SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

BIG DATA STORAGE AND PROCESSING

COURSE: IT4043E

WeaHouse: A Weather Analytics & Forecast Pipeline

Author
NGUYEN HAI DANG
NGUYEN HOANG QUAN
PHAN MINH HOA
NGUYEN VO NGOC KHUE
NGUYEN TIEN DAT

Mentor
DR. TRAN VIET TRUNG

December 2025

Contents

1. Introduction

1.1 Problem

This project focuses on designing and implementing a robust Kappa-style big data pipeline capable of supporting accurate weather forecasting and near real-time monitoring. While modern IoT weather stations generate vast amounts of environmental data, this raw data is often incomplete, noisy, and unstructured, making it unsuitable for direct predictive modeling.

The core problem is to bridge the gap between raw sensor ingestion and reliable machine learning predictions. This requires a system that not only stores data but actively repairs it – handling missing timestamps, smoothing outliers, and engineering "lag features" to create a high-quality dataset. The system aims to automate the end-to-end flow from data collection to the generation of training datasets required for forecasting models.

1.2 Motivation

With the rapid increase in environmental data generated by IoT weather stations, managing and analyzing this information requires a powerful system capable of processing high-velocity datasets. Analyzing historical weather patterns and real-time conditions can provide critical inputs for accurate forecasting, contributing to better decision-making for agriculture, energy management, and daily planning. This system aims to deliver reliable, cleaned, and timely data analysis, ensuring both forecasting models and end-users can access valuable predictive insights quickly and efficiently.

2. Dataset

2.1 Data Acquisition Process

The dataset used in this project is a comprehensive collection of historical meteorological data, specifically focused on the New York region. The data acquisition was performed using a two-stage process to ensure geographical accuracy and high-resolution time-series data:

- **Station Selection (Metadata):** We first utilized the NOAA Integrated Surface Database (ISD)[1] to identify active weather stations. This list was filtered to isolate stations located within the state of New York. From this filtered list, we extracted the precise geographic coordinates (Latitude and Longitude) for each target location.
- **Historical Data Ingestion:** Using the coordinates obtained from the ISD, we queried the Open-Meteo Historical Weather API[2]. We retrieved hourly weather records for the period spanning from **January 1, 2020, to December 31, 2024**. This approach ensured we gathered consistent, high-fidelity historical data for training our forecasting models.

2.2 Data Description

The final dataset comprises **2,104,704 records** collected from **48 distinct weather stations** across New York. Each record contains detailed atmospheric and terrestrial measurements, categorized as follows:

[1] https://www.ncei.noaa.gov/access/search/data-search/global-hourly
[2] https://open-meteo.com/en/docs/historical-weather-api

- **Location Identifiers:**
  - **Location ID:** Unique identifier for the weather station.
  - **Latitude & Longitude:** Geographic coordinates of the sensor.
  - **Elevation:** Altitude of the station above sea level.
  - **Timezone:** Local timezone context (e.g., GMT/UTC offset).
- **Atmospheric Conditions:**
  - **Temperature (2m):** Instantaneous air temperature at 2 meters above ground (°C).
  - **Apparent Temperature:** Instantaneous perceived "feels-like" temperature combining wind chill, humidity, and solar radiation (°C).
  - **Dew Point (2m):** Instantaneous dew point temperature at 2 meters (°C).
  - **Relative Humidity (2m):** Instantaneous relative humidity at 2 meters (%).
  - **Pressure (MSL & Surface):** Instantaneous atmospheric pressure reduced to mean sea level and at surface level (hPa).
- **Precipitation & Sky:**
  - **Precipitation:** Total sum of precipitation (rain, showers, snow) over the preceding hour (mm).
  - **Rain:** Sum of liquid precipitation only (rain, showers) over the preceding hour (mm).
  - **Snowfall:** Sum of snowfall amount over the preceding hour (cm).
  - **Snow Depth:** Instantaneous depth of snow on the ground (m).
  - **Cloud Cover:** Instantaneous area fraction of total cloud cover, stratified by altitude: Low (< 2 km), Mid ($2-6$ km), and High (> 6 km) (%).
  - **Weather Code:** Instantaneous WMO numeric code indicating the general weather state.
- **Wind Dynamics:**
  - **Wind Speed:** Instantaneous speed measured at 10m and 100m altitudes (km/h).
  - **Wind Direction:** Instantaneous direction of wind flow at 10m and 100m altitudes (Degrees).
  - **Wind Gusts:** Maximum wind gusts recorded during the preceding hour at 10m (km/h).
- **Soil Metrics:**
  - **Soil Temperature:** Instantaneous average temperature at varying depths: 0-7 cm, 7-28 cm, 28-100 cm, and 100-255 cm (°C).
  - **Soil Moisture:** Instantaneous volumetric soil water content (m^3/m^3) at varying depths: 0-7 cm, 7-28 cm, 28-100 cm, and 100-255 cm.

The dataset provides a comprehensive view of environmental conditions, enabling detailed analysis for applications such as time-series forecasting, anomaly detection, and the study of soil-atmosphere interactions.

3. Technologies Used and System Architecture

3.1 System Architecture

This system is architected following the **Kappa Architecture** paradigm, a modern data processing pattern that simplifies big data pipelines by treating all data as a stream. Unlike traditional architectures that maintain separate engines for batch and real-time processing, the Kappa architecture utilizes a single unified stream processing engine – **Apache Spark Structured Streaming** – to handle

both real-time ingestion and historical data transformation.

As illustrated in Figure 1, the architecture follows a unified linear flow originating from the immutable log:

- **Single Source of Truth:** All incoming data from simulated IoT sensors is ingested into Apache Kafka. This serves as the unified log, allowing the system to re-process historical data by simply rewinding the stream offsets.
- **Unified Processing Engine:** A single Spark engine consumes the data from Kafka. It splits the processing logic into two parallel streams:
  1. **Real-Time Serving Path:** Data is processed and immediately written to Cassandra to power near real-time dashboards.
  2. **Data Lakehouse Path:** Data is written to Hadoop HDFS and refined through the Medallion stages (Bronze → Silver → Gold) using micro-batch triggers managed by Apache Airflow.
- **Analytics & ML:** The Gold layer in HDFS serves as the foundation for Machine Learning tasks using **Spark ML**, while real-time metrics are visualized via **Streamlit**.

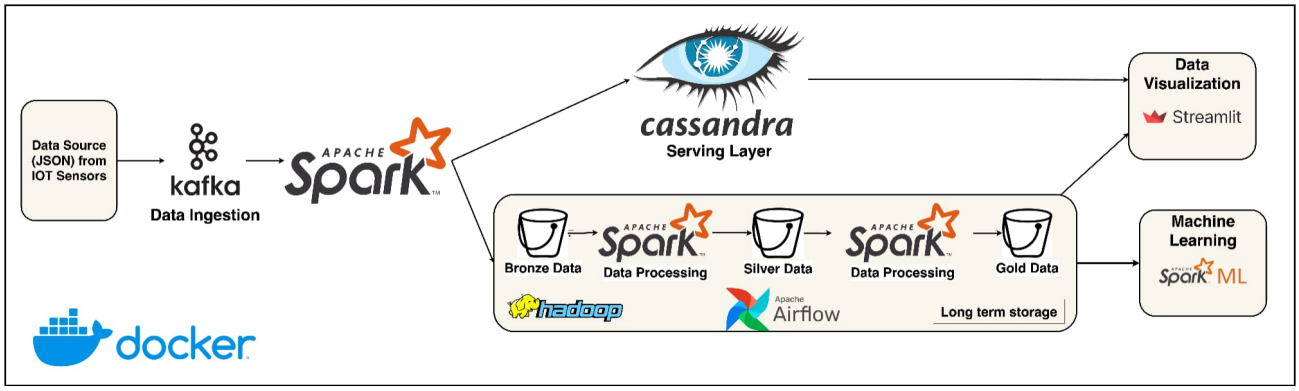

Figure 1: WeaHouse System Architecture.

3.2 Key Technologies

The system leverages a fully containerized stack. The core technologies are described below:

- **Docker**
  - **Function:** Packages code and libraries to create a consistent environment for system components.
  - **Benefits:** Enhances portability, flexibility, and eliminates environment-specific errors.
- **Apache Kafka**
  - **Function:** Acts as the central event store, ingesting streaming data from IoT sensors.
  - **Benefits:** Supports event-driven architecture, ensures zero data loss, and allows for historical replay.
- **Apache Spark**
  - **Function:** Unified engine that processes streaming data for both real-time serving and historical archiving.
  - **Benefits:** Enables low-latency analytics and complex stateful transformations on streaming data.
- **Hadoop HDFS**

  - **Function:** Distributed file system acting as the "Cold Store" for the Data Lakehouse (Parquet format).
  - **Benefits:** Provides massive scalability and high throughput for batch processing and ML training.
- **Apache Cassandra**
  - **Function:** NoSQL database acting as the "Hot Store" for real-time dashboard queries.
  - **Benefits:** Provides high write performance and sub-millisecond availability for operational data.
- **Apache Airflow**
  - **Function:** Orchestrates the workflow and schedules micro-batch triggers for Spark jobs.
  - **Benefits:** Ensures reliable pipeline execution and manages complex task dependencies.
- **Spark ML**
  - **Function:** Machine learning library used to train forecasting models on historical HDFS data.
  - **Benefits:** Scalable model training capable of processing massive datasets across the cluster.
- **Streamlit**
  - **Function:** Visualizes real-time metrics and forecast results through an interactive web interface.
  - **Benefits:** Rapid prototyping and seamless integration with Python data science stacks.

4. System Component Details

4.1 Kafka

Our Kafka cluster acts as the central ingestion layer. It is deployed with **3 broker nodes** to ensure high availability and fault tolerance. In this project, we utilize a single topic configured with **3 partitions** and a **replication factor of 3**, ensuring that data is distributed evenly and remains available even if a broker fails.

4.1.1 Kafka Cluster Configuration

Our Kafka cluster is deployed using Docker containers to ensure isolation and reproducibility. It consists of **1 Zookeeper node** for cluster coordination:

```
zookeeper:
  image: confluentinc/cp-zookeeper:7.5.0
  hostname: zookeeper
  container_name: zookeeper
  ports:
    - "2181:2181"
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181
    ZOOKEEPER_TICK_TIME: 2000
```

And 3 broker nodes. For brevity, only one broker is shown below; the other two brokers are configured identically with incremented IDs (2, 3) and ports (9093, 9094):

```
kafka1:
  image: confluentinc/cp-kafka:7.5.0
  hostname: kafka1
  container_name: kafka1
```

```
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka1:29092,PLAINTEXT_HOST://localhost:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
    KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
```

Finally, to facilitate cluster management and real-time monitoring, we deploy Kafka UI. This versatile web interface connects to all three brokers, allowing us to inspect topics, consumers, and cluster health via a dashboard:

```
kafka-ui:
  image: provectuslabs/kafka-ui:latest
  container_name: kafka-ui
  ports:
    - "8080:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: local
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka1:29092,kafka2:29092,kafka3:29092
    KAFKA_CLUSTERS_0_ZOOKEEPER: zookeeper:2181
  depends_on:
    - kafka1
    - kafka2
    - kafka3
```

Upon starting the cluster, we manually create the required topic with the specific replication configuration to ensure data durability:

```
docker exec kafka1 kafka-topics --create --if-not-exists \
    --topic weather-events \
    --bootstrap-server kafka1:29092 \
    --partitions 3 \
    --replication-factor 3 || true
```

We can verify the cluster status and topic creation via the Kafka UI dashboard. As shown below, the dashboard confirms the active status of all 3 brokers and the correct configuration of our topic.

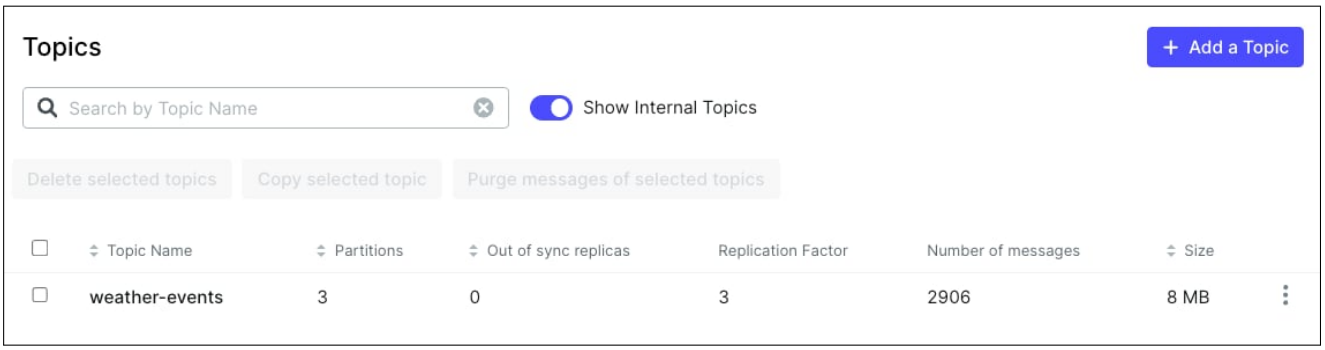

Figure 2: Kafka UI: Topic Details showing 'weather-events' with 3 partitions.

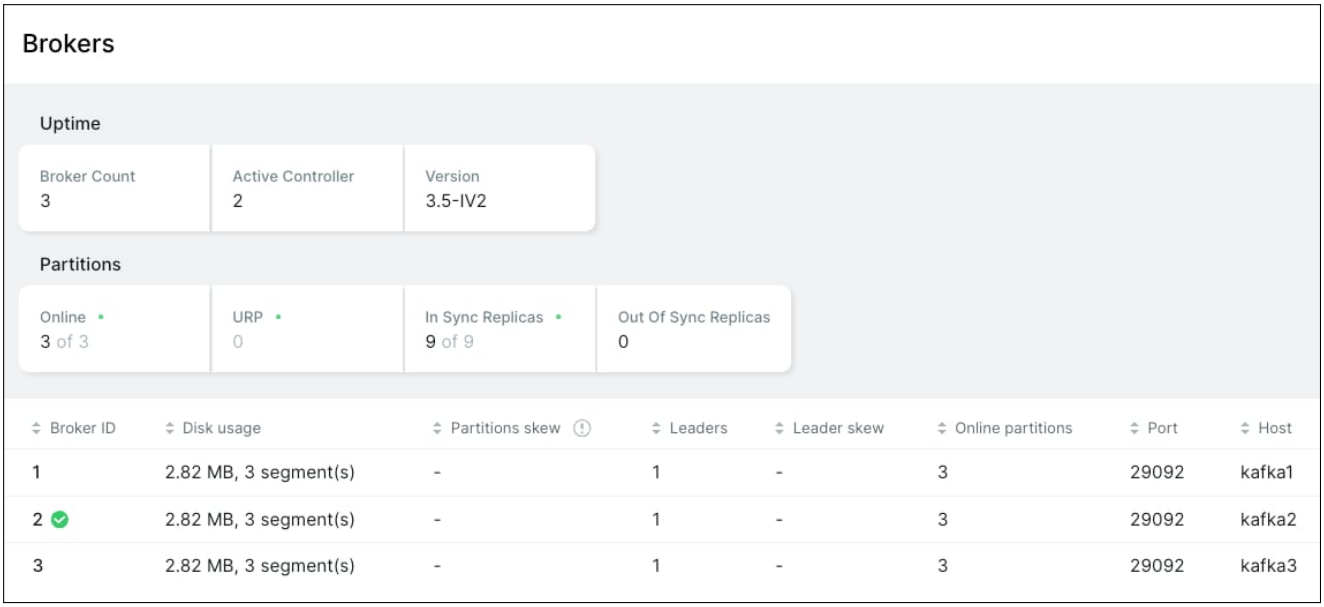

Figure 3: Kafka UI: Cluster Status confirming 3 Active Brokers.

4.1.2 Kafka Producer

Since we do not have access to a live network of physical weather stations, we developed a custom Python-based producer to simulate real-time IoT data ingestion. This component reads historical CSV data and "replays" it into the Kafka cluster, acting as a proxy for 48 distinct sensor nodes.

To rigorously test the robustness of our cleaning pipeline (the "Bronze-to-Silver" layer), the producer is engineered to intentionally inject data quality issues – mimicking the unreliability of real-world sensors as follows:

- **Chaos:** The producer randomly corrupts data based on a configurable probability ($P = 0.01$). It injects logical outliers (e.g., temperature = 999,999°C), null values or schema mismatch to verify that the Spark streaming job can handle dirty data without crashing.
- **Duplication:** Random duplicates are sent with a probability of $P = 0.01$ to test the deduplication logic in the Silver layer.

The core logic of the producer and the corruption generator is shown below:

```
# Kafka producer configuration
producer = KafkaProducer(bootstrap_servers=KAFKA_BROKERS)

# Iteratively send a row to Kafka topic
for row in csv_data:
  payload_str = json.dumps(row)
  log_msg = "Valid"

  # Randomly corrupt the payload
  if random.random() < CORRUPTION_PROBABILITY:
    payload_str, log_msg = get_corrupted_payload(row)
    print(f"ID {i}: {log_msg}")

  key = str(row['location_id']).encode('utf-8')
  value = payload_str.encode('utf-8')

  producer.send(TOPIC_NAME, key=key, value=value)

  # Randomly duplicate the message
  if random.random() < DUPLICATE_PROBABILITY:
```

```
    print(f"ID {i}: Sending DUPLICATE")
    producer.send(TOPIC_NAME, key=key, value=value)

  producer.flush()

  # Publish an article every 1 second
  time.sleep(1)
```

```
def get_corrupted_payload(row):
  row = row.copy()
  raw_json = json.dumps(row)

  # Choose a type of "chaos"
  corruption_type = random.choice(['schema_mismatch', 'outlier', 'nulls'])

  # 1. Schema Mismatch
  if corruption_type == 'schema_mismatch':
    field_to_corrupt = random.choice(NUMERIC_FIELDS)

    if field_to_corrupt in row:
      row[field_to_corrupt] = "SENSOR_ERROR_CODE_505"
      return json.dumps(row), f"Schema Mismatch ({field_to_corrupt} became String)"

  # 2. Outlier Values
  elif corruption_type == 'outlier':
    field_to_corrupt = random.choice(NUMERIC_FIELDS)

    if field_to_corrupt in row:
      # Random between 2 critical values
      val = 999999.99 if random.random() > 0.5 else -999999.99
      row[field_to_corrupt] = val
      return json.dumps(row), f"Logical Outlier ({field_to_corrupt} = {val})"

  # 3. Null Values
  elif corruption_type == 'nulls':
    valid_keys = set(NUMERIC_FIELDS).intersection(row.keys())

    if valid_keys:
      key_to_null = random.choice(list(valid_keys))
      row[key_to_null] = None
      return json.dumps(row), f"Missing Field ({key_to_null})"

  return raw_json, "Normal"
```

Beyond the intentional data corruption, the producer implementation ensures realistic streaming behavior and order preservation. Records are emitted with a 1-second delay to simulate real-time arrival latency. Crucially, messages are keyed by `location_id`, which guarantees that all weather reports from a specific station are routed to the same Kafka partition. This partitioning strategy preserves the chronological order of events for each sensor node, ensuring that downstream consumers process time-series data correctly.

4.2 HDFS

We utilize Hadoop Distributed File System (HDFS) as our "Cold Store" where all historical weather data is partitioned and stored in Parquet format. The cluster contains 1 NameNode for managing metadata, and 3 DataNodes for storing actual blocks, ensuring no data loss when a node is down.

4.2.1 Hadoop Cluster Configuration

```
namenode:
  image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
  container_name: namenode
  restart: always
  ports:
    - "9870:9870" # Web UI
    - "9000:9000" # IPC Port
  volumes:
    - namenode_data:/hadoop/dfs/name
  environment:
    - CLUSTER_NAME=weather_cluster
    - CORE_CONF_fs_defaultFS=hdfs://namenode:9000

# The 2 other nodes is the same
datanode1:
  image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
  container_name: datanode1
  restart: always
  depends_on:
    - namenode
  volumes:
    - datanode1_data:/hadoop/dfs/data
  environment:
    - SERVICE_PRECONDITION=namenode:9870
    - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
    - HDFS_CONF_dfs_replication=3
```

We can verify the health of the HDFS cluster via the NameNode web interface. As shown below, the system correctly reports the overall cluster capacity and the individual health of all data nodes.

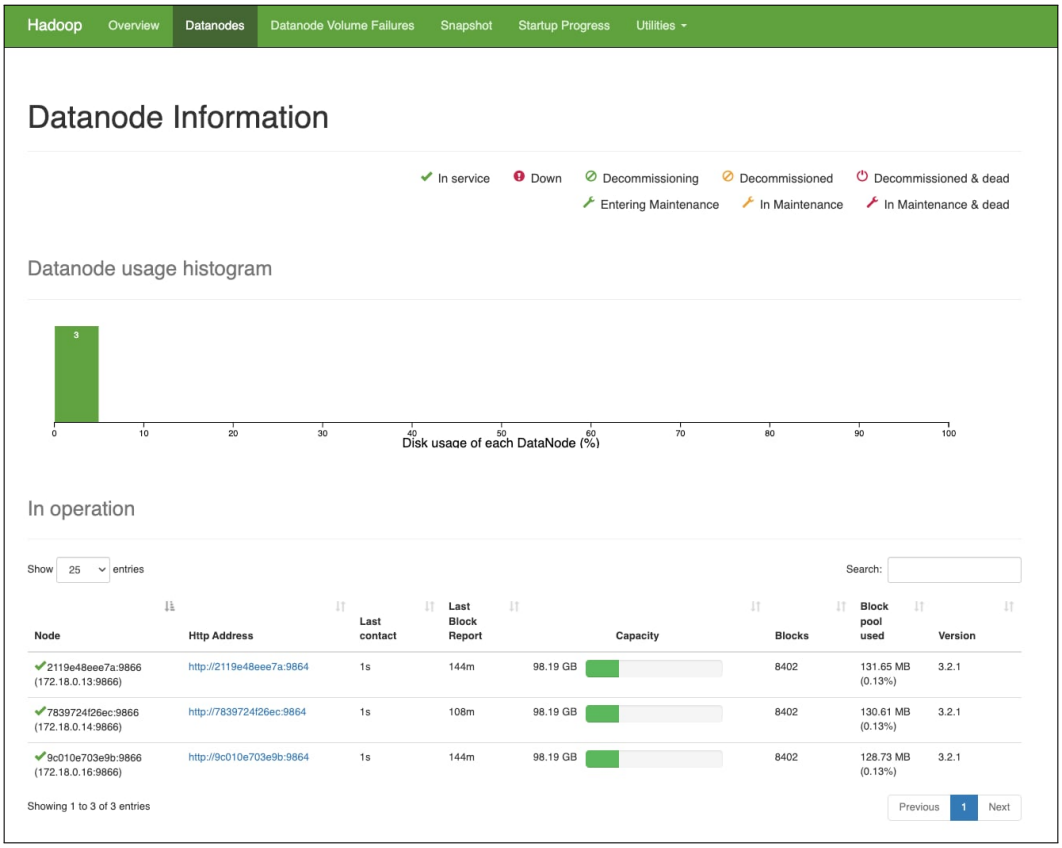

Figure 4: DataNode Information: All 3 nodes are 'In Service' and healthy.

4.3 Cassandra

To power the real-time dashboards with low-latency queries, we use Apache Cassandra. Acting as the "Hot Store," Cassandra is optimized for high-write throughput, making it ideal for ingesting the continuous stream of weather events processed by Spark.

4.3.1 Cassandra Configuration

We deploy a single-node Cassandra instance for this project, tuned for containerized performance:

```
cassandra:
  image: cassandra:4.1
  container_name: cassandra
  hostname: cassandra
  ports:
    - "9042:9042"
  environment:
    - CASSANDRA_CLUSTER_NAME=WeatherCluster
    - CASSANDRA_DC=dc1
    - CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch
    - MAX_HEAP_SIZE=512M
    - HEAP_NEWSIZE=100M
  volumes:
    - cassandra_data:/var/lib/cassandra
  healthcheck:
    test: ["CMD", "cqlsh", "-e", "describe keyspaces"]
    interval: 15s
    timeout: 10s
    retries: 10
```

4.3.2 Schema Initialization & Data Modeling

Once the Cassandra container is up, we initialize the database schema using the `cqlsh` tool. We pipe the schema file directly into the container:

```
docker exec -i cassandra cqlsh < schema.cql
```

The database schema implements a storage-friendly strategy to balance efficient disk usage with high-speed reads:

- **Static Data Separation:** We decouple constant station metadata (Latitude, Longitude, Timezone) into a dedicated `location_meta_data` table. This prevents data redundancy, ensuring that static details are not repeated for every single sensor reading in the main time-series table.
- **Time-Series Optimization:** The `raw_weather_data` table uses a composite primary key designed for range queries:
  - **Partition Key ('location_id'):** Groups all historical data for a specific station onto the same physical node.
  - **Clustering Key ('time'):** We apply `CLUSTERING ORDER BY (time DESC)` to physically store the newest records first. This ensures that queries for "Current Conditions" (e.g., `LIMIT 1`) are extremely fast, as they simply read the head of the partition without scanning history.
- **Data Lifecycle Management:** We enforce a `default_time_to_live` (TTL) of 1,296,000 seconds (approximately 15 days). This allows Cassandra to automatically expire and purge old records, ensuring storage usage remains predictable without manual maintenance.

```
-- 1. Create Keyspace
CREATE KEYSPACE IF NOT EXISTS weather_house
WITH REPLICATION = { 'class': 'SimpleStrategy', 'replication_factor': 1 };

USE weather_house;

-- 2. Metadata Table (Static Data)
-- Stores constant station details to avoid redundancy.
CREATE TABLE IF NOT EXISTS location_meta_data (
  location_id bigint PRIMARY KEY,
  latitude double,
  longitude double,
  elevation double,
  utc_offset_seconds bigint,
  timezone text,
  timezone_abbreviation text
);

-- 3. Time-Series Table (Metric Data)
CREATE TABLE IF NOT EXISTS raw_weather_data (
  location_id bigint,
  time timestamp,

  -- Quality Flag
  data_quality text,

  -- Atmospheric Metrics
  temperature_2m double,
  relative_humidity_2m double,
  dew_point_2m double,
  apparent_temperature double,
  pressure_msl double,
  surface_pressure double,
  weather_code text,

  -- Precipitation
  precipitation double,
  rain double,
  snowfall double,
  snow_depth double,

  -- Cloud Cover
  cloud_cover double,
  cloud_cover_low double,
  cloud_cover_mid double,
  cloud_cover_high double,

  -- Wind
  wind_speed_10m double,
  wind_speed_100m double,
  wind_direction_10m double,
  wind_direction_100m double,
  wind_gusts_10m double,

  -- Soil Metrics
  soil_temperature_0_to_7cm double,
  soil_temperature_7_to_28cm double,
  soil_temperature_28_to_100cm double,
  soil_temperature_100_to_255cm double,
  soil_moisture_0_to_7cm double,
  soil_moisture_7_to_28cm double,
  soil_moisture_28_to_100cm double,
```

```
  soil_moisture_100_to_255cm double,

  -- Primary Key: Partition by Location, Order by Time
  PRIMARY KEY (location_id, time)
) WITH CLUSTERING ORDER BY (time DESC)
  AND default_time_to_live = 1296000; -- 15 Days Retention
```

4.3.3 Cassandra Verification

To ensure the database is correctly provisioned and receiving data, we perform a verification process: inspecting the schema definition and validating record counts.

First, we verify the schema using the `DESCRIBE` command. This confirms that our `weather_house` keyspace is active and that the `raw_weather_data` table enforces the required `CLUSTERING ORDER` for time-series optimization.

```
(base) quan225519@weahouse-cloud-20251208-134658:~/weahouse/weather-house$ docker exec -it cassandra cqlsh -e "DESCRIBE KEYSPACE weather_house;"

CREATE KEYSPACE weather_house WITH replication = {'class': 'SimpleStrategy', 'replication_factor': '1'}  AND durable_writes = true;

CREATE TABLE weather_house.location_meta_data (
    location_id bigint PRIMARY KEY,
    elevation double,
    latitude double,
    longitude double,
    timezone text,
    timezone_abbreviation text,
    utc_offset_seconds bigint
) WITH additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND cdc = false
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND memtable = 'default'
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND extensions = {}
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';

CREATE TABLE weather_house.raw_weather_data (
    location_id bigint,
    time timestamp,
    apparent_temperature double,
    cloud_cover double,
    cloud_cover_high double,
    cloud_cover_low double,
    cloud_cover_mid double,
    data_quality text,
    dew_point_2m double,
    precipitation double,
    pressure_msl double,
    rain double,
    relative_humidity_2m double,
    snow_depth double,
    snowfall double,
    soil_moisture_0_to_7cm double,
    soil_moisture_100_to_255cm double,
    soil_moisture_28_to_100cm double,
    soil_moisture_7_to_28cm double,
    soil_temperature_0_to_7cm double,
    soil_temperature_100_to_255cm double,
    soil_temperature_28_to_100cm double,
    soil_temperature_7_to_28cm double,
    surface_pressure double,
    temperature_2m double,
    weather_code text,
    wind_direction_100m double,
    wind_direction_10m double,
    wind_gusts_10m double,
    wind_speed_100m double,
    wind_speed_10m double,
    PRIMARY KEY (location_id, time)
) WITH CLUSTERING ORDER BY (time DESC)
    AND additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND cdc = false
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND memtable = 'default'
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND extensions = {}
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';
```

Figure 5: Cassandra Schema Verification: The `DESCRIBE` command confirms the keyspace and table structure.

Second, we validate the data ingestion pipeline by querying the total row counts for both the metadata and metric tables.

```
● (base) quan225519@weahouse-cloud-20251208-134658:~/weahouse/weather-house$ docker exec -it cassandra cqlsh -e "SELECT COUNT(*) FROM weather_house.location_meta_data;"

 count
-------
    48

(1 rows)
```

Figure 6: Metadata Verification: Confirming the initialization of 48 monitoring stations.

```
● (base) quan225519@weahouse-cloud-20251208-134658:~/weahouse/weather-house$ docker exec -it cassandra cqlsh -e "SELECT COUNT(*) FROM weather_house.raw_weather_data;"

 count
-------
  2884

(1 rows)
```

Figure 7: Data Ingestion Verification: Confirming the presence of 2,884 weather records.

4.4 Spark

Following the Kappa architecture, the core processing unit is a distributed Apache Spark cluster running in **Structured Streaming** mode. This cluster acts as the consumer for the Kafka topics, responsible for both real-time ingestion into the serving layer (Cassandra) and archival into the Data Lakehouse (HDFS).

4.4.1 Custom Spark Image

To support our specific data processing logic, we cannot use the vanilla Spark image. We define a custom Dockerfile to pre-install essential Python libraries, ensuring that every Worker node has the required dependencies available at runtime.

```
# Base image matches our Spark version
FROM apache/spark:3.5.0

# Switch to root to install system dependencies
USER root

# Install Python data libraries required for our transformation logic
RUN pip install --no-cache-dir numpy pandas

# Switch back to root (or default user) for execution
USER root
```

4.4.2 Spark Cluster Configuration

The cluster is configured in Standalone mode with a Master-Slave architecture. We deploy 1 Master node for resource management and 3 Worker nodes. Below is the configuration for the Master and the first Worker node. The other two workers are configured identically.

```
spark-master:
  build: src/spark
  container_name: spark-master
  hostname: spark-master
  ports:
    - "8081:8080"  # Spark Master Web UI
    - "7077:7077"  # Cluster Connection Port
  volumes:
```

```
    - ./src:/opt/spark/src
  command: /opt/spark/bin/spark-class org.apache.spark.deploy.master.Master

# Worker 1 Configuration (Workers 2 and 3 are identical)
spark-worker-1:
  build: src/spark
  container_name: spark-worker-1
  hostname: spark-worker-1
  depends_on:
    - spark-master
  volumes:
    - ./src:/opt/spark/src
  environment:
    SPARK_WORKER_CORES: 1
    SPARK_WORKER_MEMORY: 2G
  command: /opt/spark/bin/spark-class org.apache.spark.deploy.worker.Worker spark://spark-master
    :7077
```

4.4.3 Spark Structured Streaming

The core pipeline is a PySpark application that consumes the `weather-events` Kafka topic. To balance the need for historical accuracy and low-latency availability, we utilize the `foreachBatch` execution model. This approach allows us to fork the stream into two distinct paths within a single micro-batch:

- **Cold Path:** Raw data is written immediately to HDFS in Parquet format. This preserves the original data state – including potential corruptions – ensuring a "replayable" archive for future model training.
- **Hot Path:** Data is cleaned, imputed, and written to Cassandra. Crucially, this step performs a normalized write: static metadata (e.g., timezone, elevation) is extracted and written to the `location_meta_data` table, while dynamic metrics are written to the `raw_weather_data` table.

a. Configuration and Constants

We define ‘META_COLS‘ to identify static attributes and ‘PHYSICAL_LIMITS‘ to enforce data quality thresholds (e.g., wind speed cannot exceed 450 km/h).

```
# Columns belonging to the static metadata table
META_COLS = [
  "location_id", "latitude", "longitude", "elevation",
  "utc_offset_seconds", "timezone", "timezone_abbreviation"
]

PHYSICAL_LIMITS = [
  # Atmospheric
  ("temperature_2m", -95, 65),
  ("relative_humidity_2m", 0, 105),
  ("pressure_msl", 850, 1100),
  # Wind
  ("wind_speed_10m", 0, 450),
  ("wind_gusts_10m", 0, 450),
  # ... (Full list of soil, cloud, and precip limits applied)
]

def get_spark_session():
  return SparkSession.builder \
```

```
    .appName("WeatherHDFSWrite") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:9000") \
    .config("spark.cassandra.connection.host", "cassandra") \
    .config("spark.cassandra.connection.port", "9042") \
    .config("spark.cores.max", "1") \
    .config("spark.executor.memory", "1G") \
    .config("spark.driver.memory", "512m") \
    .getOrCreate()
```

b. The Micro-Batch Processing Logic

The `process_batch` function executes the logic for every micro-batch (triggered every 30 seconds). We explicitly cache the dataframe to optimize performance across the multiple write actions.

1. **Cold Store:** The raw batch is appended to HDFS partitioned by `location_id`.
2. **Transformation:** We filter "Physical Limit" outliers by converting them to `NULL`.
3. **Imputation:** We use Window functions (LOCF - Last Observation Carried Forward) to fill gaps created by sensor failures or outlier filtering.
4. **Split Write (Cassandra):**
   - **Metadata:** We extract distinct 'META_COLS' and write to the metadata table.
   - **Metrics:** We drop the metadata columns and write the remaining time-series data to the metrics table.

```
def process_batch(batch_df, batch_id):
  if batch_df.isEmpty(): return

  print(f"Processing Batch ID: {batch_id} with {batch_df.count()} records")
  batch_df.cache()

  # 1. COLD PATH: Archive Raw Data to HDFS (Bronze Layer)
  batch_df.write.mode("append").partitionBy("location_id") \
    .parquet("hdfs://namenode:9000/weather/bronze")

  # 2. HOT PATH: Cleaning & Imputation
  processing_df = batch_df.dropDuplicates(["location_id", "time"])

  # Nullify Outliers
  for col_name, min_val, max_val in PHYSICAL_LIMITS:
    if col_name in processing_df.columns:
      processing_df = processing_df.withColumn(col_name,
        when((col(col_name) < min_val) | (col(col_name) > max_val), lit(None))
        .otherwise(col(col_name))
      )

  # Apply LOCF (Last Observation Carried Forward)
  w = Window.partitionBy("location_id").orderBy("time")
  imputed_df = processing_df.withColumn("data_quality", lit("OK"))

  for col_name, _, _ in PHYSICAL_LIMITS:
    if col_name in imputed_df.columns:
      last_val = last(col(col_name), ignorenulls=True).over(w)
      # Update quality flag and fill nulls
      imputed_df = imputed_df.withColumn("data_quality",
        when(col(col_name).isNull() & last_val.isNotNull(), lit("IMPUTED"))
        .otherwise(col("data_quality"))
      ).withColumn(col_name, coalesce(col(col_name), last_val))

  # 3. HOT PATH WRITE: Cassandra
```

```
  try:
    cassandra_prep_df = imputed_df.withColumn("time", to_timestamp(col("time")))

    # Write 1: Static Metadata (Normalized)
    cassandra_prep_df.select(*META_COLS).distinct().write \
      .format("org.apache.spark.sql.cassandra") \
      .options(table="location_meta_data", keyspace="weather_house") \
      .mode("append").save()

    # Write 2: Time-Series Metrics (Simplified)
    cols_to_drop = [c for c in META_COLS if c != "location_id"]
    cassandra_prep_df.drop(*cols_to_drop).write \
      .format("org.apache.spark.sql.cassandra") \
      .options(table="raw_weather_data", keyspace="weather_house") \
      .mode("append").save()

  except Exception as e:
    print(f"Cassandra Error: {e}")

  batch_df.unpersist()
```

c. Streaming Execution

The main entry point initiates the stream from `earliest` offsets to ensure no data is missed during restarts.

```
def main():
  spark = get_spark_session()
  spark.sparkContext.setLogLevel("WARN")

  # Read from Kafka
  kafka_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka1:29092,kafka2:29092,kafka3:29092") \
    .option("subscribe", "weather-events") \
    .option("startingOffsets", "earliest") \
    .load()

  # Parse JSON
  raw_parsed_df = kafka_df.select(
    from_json(col("value").cast("string"), weather_schema).alias("data")
  ).select("data.*")

  # Start Stream
  query = raw_parsed_df.writeStream \
    .foreachBatch(process_batch) \
    .option("checkpointLocation", "hdfs://namenode:9000/weather/checkpoints/streaming") \
    .trigger(processingTime="30 seconds") \
    .start()

  query.awaitTermination()
```

4.4.4 Spark Medallion Jobs

While the "Hot Path" feeds Cassandra immediately after light preprocessing, our "Cold Path" requires a more robust cleaning and aggregation pipeline. We implement this using two separate Spark jobs that progressively refine the data.

a. Bronze to Silver

This job reads from the Bronze layer in HDFS. Its primary goal is to resolve data quality issues introduced by the producer's "Chaos Engineering" module.

The transformation logic includes:

- **Deduplication:** Removes duplicate records based on `location_id` and `time`.
- **Outlier Removal:** Checks values against strict physical limits (e.g., Wind Speed < 450 km/h). Violations are nullified.
- **Temporal Imputation:** Uses window functions to fill missing values (Nulls) by interpolating between the previous and next valid records (Forward/Backward Fill).
- **Circular Interpolation:** Special handling for Wind Direction (degrees) to correctly interpolate values (e.g., averaging 350° and 10° should yield 0°, not 180°).

Key Methods Used:

- **foreachBatch:** This method allows applying Batch processing logic, such as Window functions, Joins, or writing data to multiple destinations, to each micro-batch of a streaming DataFrame. This effectively overcomes the limitations of Spark Structured Streaming when handling complex operations that are not natively supported on continuous data streams.
- **coalesce:** This function is used to select the first non-null value from a list of provided columns or value. In this pipeline, `coalesce` plays a critical role in prioritizing the retention of original data, replacing it with an interpolated value only when the original record is missing (null).

```
# ... (Imports and SparkSession setup omitted for brevity) ...

# 1. READ BRONZE STREAM
bronze_df = spark.readStream.schema(weather_schema).parquet("/weather/bronze")

PHYSICAL_LIMITS = [
  ("temperature_2m", -95, 65), ("wind_speed_10m", 0, 450),
  ("precipitation", 0, 400),   ("pressure_msl", 850, 1100),
  # ... (full list of 25+ sensors defined here)
]

def process_and_split(batch_df, batch_id):
  if batch_df.isEmpty(): return

  print(f"--- Processing Batch {batch_id} ---")
  batch_df.cache()

  # STEP 1: DEDUPLICATE & NULLIFY OUTLIERS
  cleaned_df = batch_df.dropDuplicates(["location_id", "time"])

  # Nullify values outside physical limits
  for col_name, min_val, max_val in PHYSICAL_LIMITS:
    if col_name in cleaned_df.columns:
      cleaned_df = cleaned_df.withColumn(col_name,
        when(
          (col(col_name) < min_val) | (col(col_name) > max_val),
          lit(None)
        ).otherwise(col(col_name))
      )

```

```
# STEP 2: TEMPORAL IMPUTATION
# Prepare Time Columns for weighted interpolation
df_with_ts = cleaned_df.withColumn("event_ts", col("time").cast("timestamp")) \
  .withColumn("event_unix", unix_timestamp("event_ts")) \
  .withColumn("imputed_flag", lit(False))

w = Window.partitionBy("location_id").orderBy("event_ts")
w_future = w.rowsBetween(0, Window.unboundedFollowing)

# Calculate Forward Fill (ff) and Backward Fill (bf) for ALL linear columns
linear_cols = ["temperature_2m", "pressure_msl", "precipitation", "wind_speed_10m", ...]

for c in linear_cols:
  # Get last valid value and next valid value
  ff_val = last(col(c), ignorenulls=True).over(w)
  bf_val = first(col(c), ignorenulls=True).over(w_future)

  # Get timestamps for weighting
  ff_time = last(when(col(c).isNotNull(), col("event_unix")), ignorenulls=True).over(w)
  bf_time = first(when(col(c).isNotNull(), col("event_unix")), ignorenulls=True).over(w_future)

  # Apply Weighted Linear Interpolation Formula
  interpolated = when(col(c).isNotNull(), col(c)) \
    .otherwise(
      when(ff_val.isNotNull() & bf_val.isNotNull() & (ff_time != bf_time),
        ff_val + (col("event_unix") - ff_time) * (bf_val - ff_val) / (bf_time - ff_time)
      ).otherwise(coalesce(ff_val, bf_val))
    )

  # Flag and Apply
  df_with_ts = df_with_ts.withColumn("imputed_flag",
    when(col(c).isNull() & interpolated.isNotNull(), lit(True))
    .otherwise(col("imputed_flag"))
  ).withColumn(c, interpolated)

# Standard linear averaging fails for degrees
for dir_col in ["wind_direction_10m", "wind_direction_100m"]:
  dir_was_null = col(dir_col).isNull()

  # Decompose degrees to Sin/Cos vectors
  df_with_ts = df_with_ts.withColumn(f"{dir_col}_rad", radians(col(dir_col))) \
    .withColumn(f"{dir_col}_x", cos(col(f"{dir_col}_rad"))) \
    .withColumn(f"{dir_col}_y", sin(col(f"{dir_col}_rad")))

  # Interpolate Vectors independently
  for component in ["_x", "_y"]:
    comp_col = f"{dir_col}{component}"
    ff_val = last(col(comp_col), ignorenulls=True).over(w)
    bf_val = first(col(comp_col), ignorenulls=True).over(w_future)

    # Simple Coalesce is sufficient for vector stability here
    interpolated = coalesce(col(comp_col), ff_val, bf_val, lit(0.0))
    df_with_ts = df_with_ts.withColumn(comp_col, interpolated)

  # Recompose Vectors back to Degrees (0-360)
  df_with_ts = df_with_ts.withColumn(dir_col,
    (degrees(atan2(col(f"{dir_col}_y"), col(f"{dir_col}_x"))) + 360) % 360
  )

  # Flag if imputed
  df_with_ts = df_with_ts.withColumn("imputed_flag",
    when(dir_was_null & col(dir_col).isNotNull(), lit(True))
```

```
      .otherwise(col("imputed_flag"))
    ).drop(f"{dir_col}_rad", f"{dir_col}_x", f"{dir_col}_y")

  # STEP 3: SPLIT AND WRITE
  final_df = df_with_ts.withColumn("data_quality",
    when(col("imputed_flag"), lit("IMPUTED")).otherwise(lit("OK"))
  )

  # Write Dimension (Location Metadata)
  final_df.select("location_id", "latitude", "longitude", "timezone") \
    .distinct().write.mode("append").parquet("/weather/silver/dim_location")

  # Write Fact (Weather Measurements)
  final_df.select(*fact_cols) \
    .write.mode("append").partitionBy("location_id") \
    .parquet("/weather/silver/fact_weather")

  batch_df.unpersist()

# Main Execution
query = bronze_df.writeStream \
  .foreachBatch(process_and_split) \
  .option("checkpointLocation", "/weather/checkpoints/silver_simplified") \
  .trigger(availableNow=True) \
  .start()

query.awaitTermination()
```

Listing 1: Bronze to Silver ETL

b. Silver to Gold

The Gold layer is designed for Batch Analytics and Machine Learning. This job runs on a schedule to aggregate the high-frequency sensor data into meaningful time buckets. It performs two key tasks:

1. **Aggregations:** Summarizes weather metrics into Daily, Weekly, and Monthly tables.
2. **ML Feature Engineering:** Generates features specifically for forecasting models:
   - **Cyclical Time:** Converts Day-of-Year into Sin/Cos waves to capture seasonality.
   - **Lag Features:** Captures "yesterday's" weather (e.g., `lag_1d_avg_temp`).
   - **Rolling Windows:** Calculates 7-day and 30-day moving averages to smooth out noise.

```
# ... (Imports and SparkSession setup omitted) ...

def main():
  # Read Silver Layer (Fact + Dimension Join)
  fact_df = spark.read.parquet("/weather/silver/fact_weather")
  dim_df = spark.read.parquet("/weather/silver/dim_location")
  full_df = fact_df.join(broadcast(dim_df), "location_id") \
    .withColumn("date", to_date(col("event_time")))

  # PART 1: AGGREGATIONS (Daily, Weekly, Monthly)
  # Helper to calculate Avg, Max, Min, Sum for all sensors
  def write_aggregation(df, group_cols, output_path):
    agg_df = df.groupBy(*group_cols).agg(
      round(avg("temperature_2m"), 2).alias("avg_temp_c"),
      round(sum("precipitation"), 2).alias("total_precip_mm"),
      round(max("wind_gusts_10m"), 2).alias("max_wind_gust_kmh"),
      # ... (20+ other metrics) ...
      count("event_time").alias("hours_recorded")
    )
```

```
    agg_df.write.mode("overwrite").parquet(output_path)
    return agg_df

# Generate Summaries
daily_df = write_aggregation(full_df, ["location_id", "date"],
  "/weather/gold/daily_summary")
write_aggregation(full_df.withColumn("week_start", trunc(col("date"), "week")),
  ["location_id", "week_start"], "/weather/gold/weekly_summary")

# PART 2: ML FEATURE ENGINEERING
# 2.1 Cyclical Time Features (Seasonality)
ml_features_df = daily_df \
  .withColumn("day_of_year", dayofyear("date")) \
  .withColumn("doy_sin", sin(2 * 3.14159 * col("day_of_year") / 365)) \
  .withColumn("doy_cos", cos(2 * 3.14159 * col("day_of_year") / 365))

# 2.2 Lags & Rolling Averages (Trends)
w_loc = Window.partitionBy("location_id").orderBy("date")
w_rolling_7d = w_loc.rowsBetween(-6, 0) # 7-day moving window

for metric in ["avg_temp_c", "total_precip_mm", "avg_pressure_msl_hpa"]:
  ml_features_df = ml_features_df \
    .withColumn(f"lag_1d_{metric}", lag(metric, 1).over(w_loc)) \
    .withColumn(f"rolling_7d_{metric}", avg(metric).over(w_rolling_7d))

ml_features_df.write.mode("overwrite").parquet("/weather/gold/ml_features")
```

Listing 2: Silver to Gold ETL

4.4.5 Spark ML

Leveraging the feature-rich "Gold" layer, we implement a supervised machine learning pipeline to forecast future weather conditions. A key design choice in this architecture is **Localized Learning**: instead of training one massive global model, we train distinct Random Forest models for each weather station. This allows the system to capture unique microclimate patterns (e.g., a coastal station behaves differently than a mountain station).

a. Model Training Strategy

The training pipeline implements a **Localized Modeling** approach, recognizing that weather patterns are highly dependent on specific microclimates (e.g., a coastal station behaves differently from a mountainous one). Instead of a single global model, the system iterates through every active station to train a dedicated suite of models. The process involves three key stages:

1. **Feature Engineering & Vector Assembly:** We construct a dense feature vector combining historical metrics and temporal signals. We apply Cyclical Time Encoding to the `day_of_year` using sine and cosine transformations:

$$x_{\sin} = \sin\left(\frac{2\pi \cdot d}{365}\right), \quad x_{\cos} = \cos\left(\frac{2\pi \cdot d}{365}\right)$$

   This ensures the model understands that December 31st (Day 365) is temporally adjacent to January 1st (Day 1).
2. **Label Generation (Supervised Learning):** To frame the forecasting problem as supervised learning, we employ Spark Window functions. The `lead()` function shifts the time-series data back-

ward by one interval (24 hours), aligning the features at time T with the target variable at $T+1$ (e.g., predicting tomorrow's temperature based on today's conditions).

3. **Ensemble Learning via Random Forest:** For each station, we train four distinct models to handle different data types:
   - **Regressors (Temperature, Humidity):** We use Random Forest Regressors to predict continuous variables. Random Forests are chosen for their ability to handle non-linear relationships and resistance to overfitting on noisy sensor data.
   - **Classifiers (Rain, Snow):** We use Random Forest Classifiers to predict the binary probability of precipitation events.

```
# ... (Imports and SparkSession setup omitted) ...

def main():
  print("STARTING: Localized Weather Model Training")

  # 1. LOAD FEATURES (From Gold Layer)
  # Reads the pre-computed daily features created by the Silver-to-Gold job
  df = spark.read.parquet("/weather/gold/ml_features")

  # 2. LABEL GENERATION (Time-Shifting)
  # We want to predict T+1 (Tomorrow) using features from T (Today)
  w = Window.partitionBy("location_id").orderBy("date")

  train_df = df.withColumn("label_temp", lead("avg_temp_c", 1).over(w)) \
    .withColumn("label_humid", lead("avg_humidity", 1).over(w)) \
    .withColumn("label_rain", when(lead("total_precip_mm", 1).over(w) > 0, 1.0).otherwise(0.0)) \
    .withColumn("label_snow", when(lead("total_snow_cm", 1).over(w) > 0, 1.0).otherwise(0.0))

  # Drop the last row of each partition (which has no target/label)
  train_df = train_df.na.drop(subset=["label_temp"])
  train_df.cache()

  # 3. VECTOR ASSEMBLY
  # Combine numerical features and cyclical date encodings into a single vector
  feature_cols = [
    "doy_sin", "doy_cos",           # Cyclical Time Features
    "avg_pressure_msl_hpa",         # Barometric Pressure
    "avg_temp_c", "total_precip_mm", "avg_humidity", # Recent conditions
    "rolling_7d_avg_temp_c", "rolling_7d_total_precip_mm" # Trends
  ]
  assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")

  # 4. LOCALIZED TRAINING LOOP
  # Iterate through every unique station ID to train a custom model
  stations = [row.location_id for row in train_df.select("location_id").distinct().collect()]
  print(f" -> Found {len(stations)} active stations. Starting training...")

  for sid in stations:
    station_data = train_df.filter(col("location_id") == sid)

    # Safety check: Need at least a few days of history to train
    if station_data.count() < 10: continue

    # Split Data (80% Train, 20% Validation)
    train, test = station_data.randomSplit([0.8, 0.2], seed=42)

    def train_and_save(model_type, label_col, model_name):
      if model_type == "regressor":
        rf = RandomForestRegressor(featuresCol="features", labelCol=label_col, numTrees=20,
```

```
    maxDepth=8)
      else:
        rf = RandomForestClassifier(featuresCol="features", labelCol=label_col, numTrees=20,
    maxDepth=8)

      pipeline = Pipeline(stages=[assembler, rf])
      model = pipeline.fit(train)

      # Persistence: Save model to HDFS under unique station path
      path = f"hdfs://namenode:9000/weather/models/station_{sid}/{model_name}"
      model.write().overwrite().save(path)

    try:
      # Train 4 distinct models per station
      train_and_save("regressor", "label_temp",  "rf_temp_model")
      train_and_save("regressor", "label_humid", "rf_humid_model")
      train_and_save("classifier", "label_rain", "rf_rain_model")
      train_and_save("classifier", "label_snow", "rf_snow_model")
      print(f"    [OK] Station {sid}: All models updated.")
    except Exception as e:
      print(f"    [FAIL] Station {sid}: {e}")

  train_df.unpersist()
```

Listing 3: Localized Model Training (train_model.py)

b. Forecasting

The prediction script is executed daily to generate weather forecasts for the upcoming 24 hours. Unlike real-time streaming jobs, this batch process reads from the **Silver Layer (HDFS)** rather than the operational database (Cassandra) to ensure analytical efficiency and minimize load on the serving layer. It dynamically loads the specific models associated with each station, ensuring that predictions are tailored to the local environment. The inference process follows these steps:

1. **Data Extraction & Optimization:** The script reads historical facts from the Silver layer, applying a filter to extract only the last 30 days of data. This avoids full-table scans and ensures the 7-day rolling window features can be computed efficiently.
2. **Feature Construction:** Raw hourly data is aggregated into daily statistics, and rolling window features (e.g., 7-day average temperature) are computed to create "Today's" feature vector.
3. **Multi-Model Chaining:** The script loads and executes the four station-specific models in sequence:
   - **Temperature & Humidity:** (Regressors) Predict specific continuous values.
   - **Rain & Snow:** (Classifiers) Predict binary probabilities (Yes/No).
4. **Result Consolidation:** All predictions are merged into a unified forecast table and saved to the **Gold Layer** for consumption by the dashboard.

```
# ... (Imports and SparkSession setup omitted) ...

def main():
  print("WEATHER FORECAST: SILVER TO GOLD")

  # 1. READ SILVER DATA (Optimization: Recent Data Only)
  # Read from HDFS (Columnar Parquet) instead of Cassandra for speed
  silver_df = spark.read.parquet("/weather/silver/fact_weather")

  # Filter: Keep only data from (Max Date - 30 Days) onwards
```

```
# This ensures we have enough history for rolling windows without scanning everything
max_ts = silver_df.agg(max_("event_time")).collect()[0][0]
cutoff_ts = date_sub(lit(max_ts), 30)
recent_df = silver_df.filter(col("event_time") >= cutoff_ts)

# 2. FEATURE ENGINEERING
# Aggregate Hourly -> Daily
daily_df = recent_df.withColumn("date", to_date(col("event_time"))) \
  .groupBy("location_id", "date").agg(
    round(avg("temperature_2m"), 2).alias("avg_temp_c"),
    round(sum("precipitation"), 2).alias("total_precip_mm"),
    round(avg("relative_humidity_2m"), 2).alias("avg_humidity"),
    round(avg("pressure_msl"), 2).alias("avg_pressure_msl_hpa")
  )

# Calculate Rolling Features (7-day history)
w_loc = Window.partitionBy("location_id").orderBy("date")
w_rolling_7d = w_loc.rowsBetween(-6, 0)

features_df = daily_df \
  .withColumn("day_of_year", dayofyear("date")) \
  .withColumn("doy_sin", sin(2 * 3.14159 * col("day_of_year") / 365)) \
  .withColumn("doy_cos", cos(2 * 3.14159 * col("day_of_year") / 365)) \
  .withColumn("rolling_7d_avg_temp_c", avg("avg_temp_c").over(w_rolling_7d)) \
  .withColumn("rolling_7d_total_precip_mm", avg("total_precip_mm").over(w_rolling_7d))

# Select only the MOST RECENT complete day for prediction
latest_dates = features_df.groupBy("location_id").agg(max_("date").alias("max_date"))

current_weather = features_df.join(latest_dates,
  (features_df.location_id == latest_dates.location_id) &
  (features_df.date == latest_dates.max_date)
).select(features_df["*"]).na.fill(0)

current_weather.cache()

# 3. DYNAMIC MODEL LOADING & INFERENCE
active_stations = [row.location_id for row in current_weather.select("location_id").distinct().
  collect()]
forecast_dfs = []

for sid in active_stations:
  station_data = current_weather.filter(col("location_id") == sid)
  base_path = f"hdfs://namenode:9000/weather/models/station_{sid}"

  try:
    # Load all 4 specialized models for this specific station
    m_temp = PipelineModel.load(f"{base_path}/rf_temp_model")
    m_humid = PipelineModel.load(f"{base_path}/rf_humid_model")
    m_rain = PipelineModel.load(f"{base_path}/rf_rain_model")
    m_snow = PipelineModel.load(f"{base_path}/rf_snow_model")

    # Chain predictions sequentially
    p1 = m_temp.transform(station_data).withColumnRenamed("prediction", "pred_temp")
    p2 = m_humid.transform(p1).withColumnRenamed("prediction", "pred_humid")
    p3 = m_rain.transform(p2).withColumnRenamed("prediction", "pred_rain_prob")
    final_pred = m_snow.transform(p3).withColumnRenamed("prediction", "pred_snow_prob")

    forecast_dfs.append(final_pred.drop("features", "rawPrediction", "probability"))
  except Exception as e:
    print(f"Failed to predict for Station {sid}: {e}")
    continue
```

```
72
73  # 4. CONSOLIDATE AND SAVE TO GOLD LAYER
74  if not forecast_dfs: return
75
76  final_df = reduce(lambda df1, df2: df1.union(df2), forecast_dfs)
77
78  forecast_df = final_df.select(
79    col("location_id"),
80    col("date").alias("based_on_date"),
81    date_add(col("date"), 1).alias("forecast_date"), # Predict for Tomorrow
82    round(col("pred_temp"), 1).alias("pred_temp_c"),
83    round(col("pred_humid"), 1).alias("pred_humidity"),
84    when(col("pred_rain_prob") >= 0.5, "YES").otherwise("NO").alias("pred_is_rain"),
85    when(col("pred_snow_prob") >= 0.5, "YES").otherwise("NO").alias("pred_is_snow")
86  ).orderBy("location_id")
87
88  output_path = "/weather/gold/weather_forecast"
89  forecast_df.write.mode("overwrite").parquet(output_path)
90  print(f" -> Forecasts saved to {output_path}")
```

Listing 4: Inference Script (predict_weather.py)

4.4.6 Spark Verification

To confirm the successful deployment of our distributed processing engine and the execution of our Medallion jobs, we perform verification at two levels: the infrastructure level (Spark Cluster) and the storage level (HDFS Data Lakehouse).

a. Cluster Health

We access the Spark Master Web UI to ensure that resources are allocated correctly according to our `docker-compose` configuration.

As shown in Figure 8, the cluster status is "ALIVE" with 3 active Workers. Each worker is correctly provisioned with 1 Core and 2 GB of memory, confirming that our distributed processing environment is ready to handle parallel tasks.

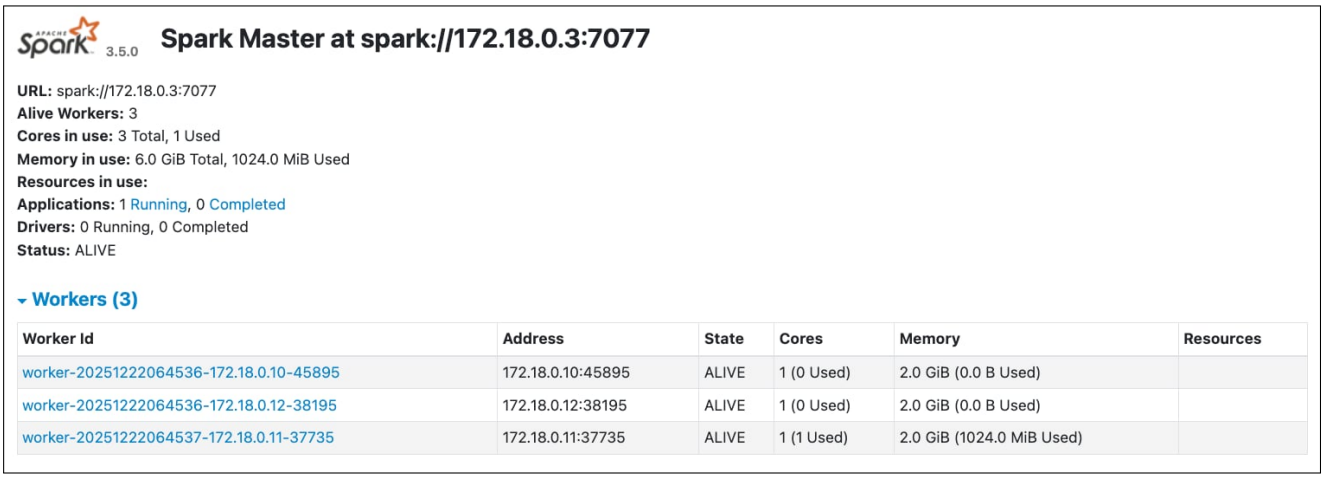

Figure 8: Spark Master UI: Verification of 3 active workers and resource allocation.

b. HDFS Structure

Next, we inspect the Hadoop File System to verify that the Spark Structured Streaming and Batch jobs are correctly writing data to the appropriate layers.

Figure 9 shows the root `/weather` directory, confirming the creation of the standard Medallion zones (`bronze`, `silver`, `gold`) along with the `checkpoints` directory for streaming state management and the `models` directory for ML artifacts.

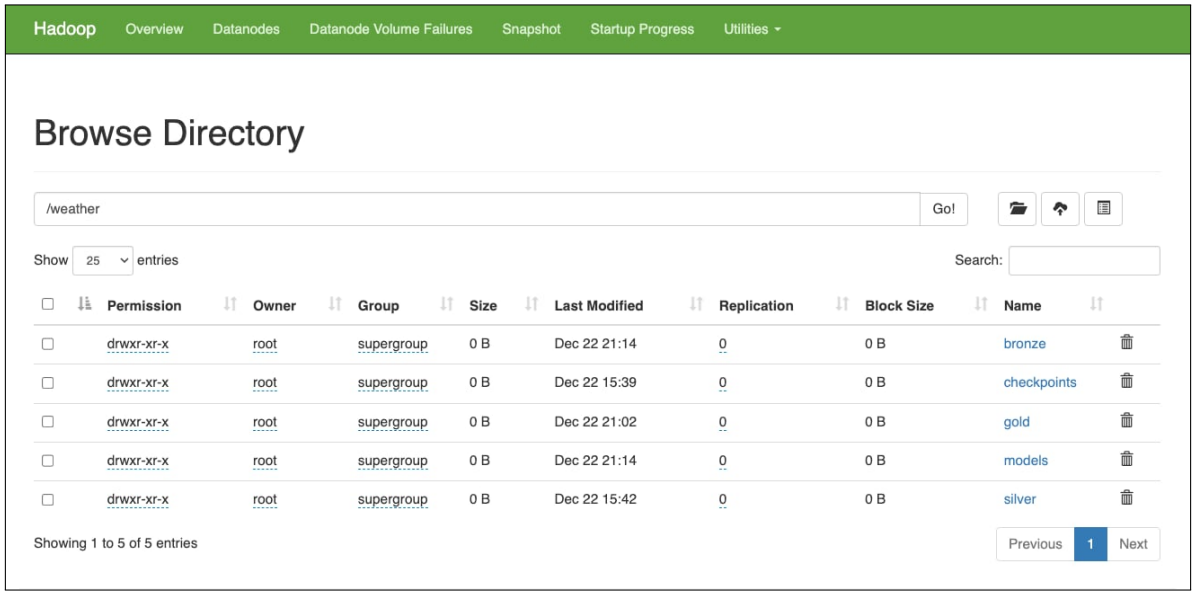

Figure 9: HDFS Root: Verification of the Medallion Architecture folder structure.

c. Layer-Specific Verification

- **Bronze Layer:** Figure 10 confirms that the streaming job is successfully archiving raw data. The presence of `location_id=x` folders verifies that our **partitioning strategy** is working, which is critical for query performance.
- **Silver Layer:** Figure 11 shows the successful split of data into a Star Schema. The existence of `fact_weather` and `dim_location` proves that the "Bronze-to-Silver" ETL job is correctly separating dimensions from facts.
- **Gold Layer:** Figure 12 verifies the output of our batch analytics and machine learning jobs. We see the time-based aggregations (`daily`, `weekly`) and the `weather_forecast` folder, confirming that the inference pipeline has successfully run and saved predictions.

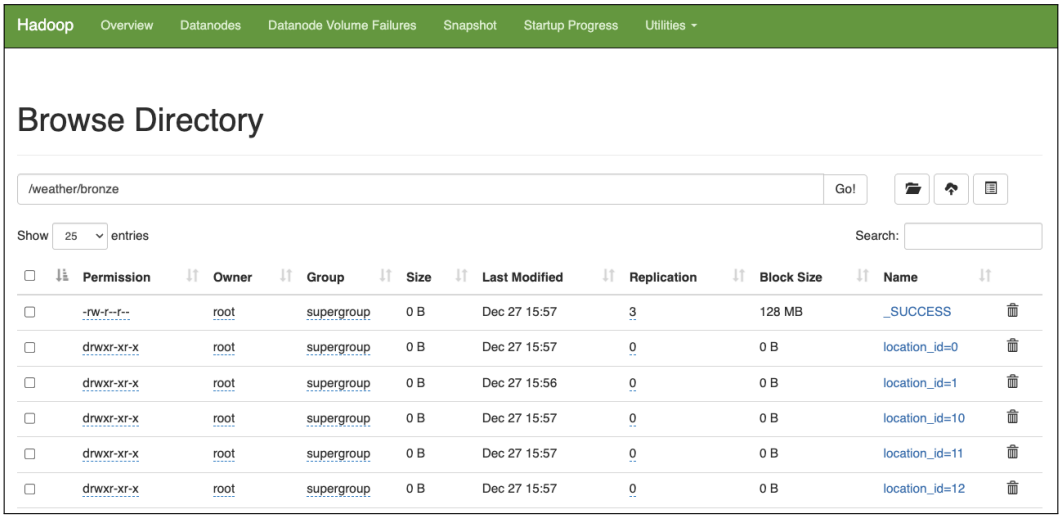

Figure 10: HDFS Bronze Layer: Verification of location-based partitioning.

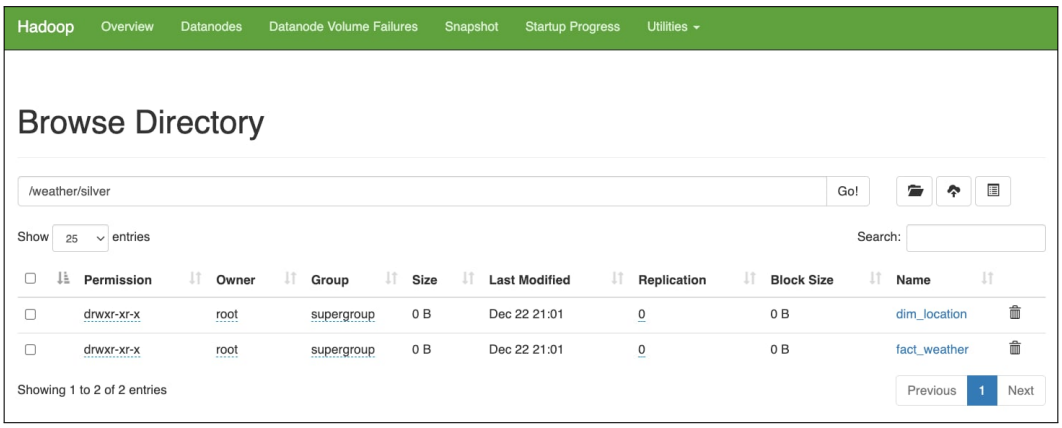

Figure 11: HDFS Silver Layer: Successful Dimension/Fact table split.

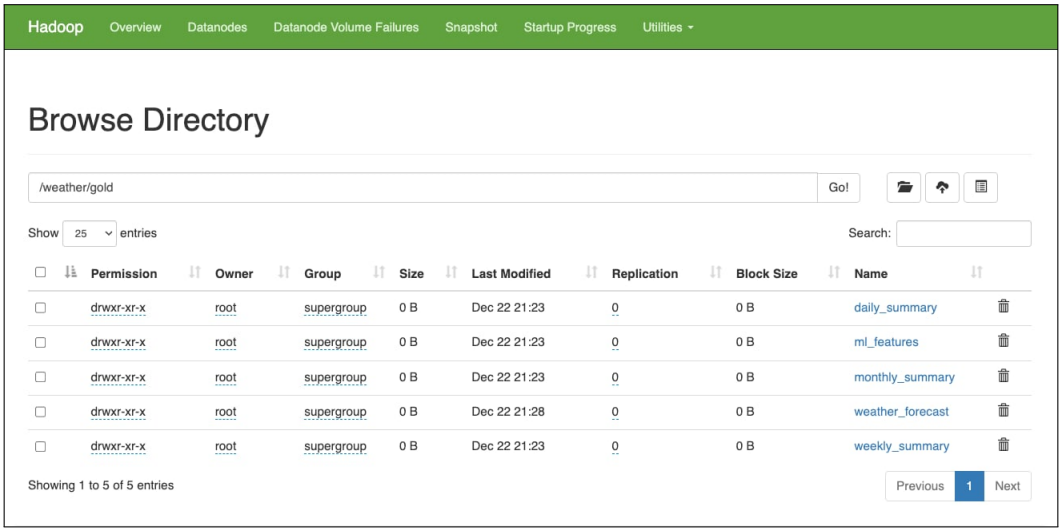

Figure 12: HDFS Gold Layer: Aggregations and Machine Learning forecasts.

4.5 Airflow

To manage the dependencies between our Batch ETL jobs and Machine Learning pipelines, we utilize **Apache Airflow**. Airflow ensures that tasks run in the correct order and on a reliable schedule.

4.5.1 Airflow Configuration

A unique challenge in a containerized environment is allowing the Orchestrator (Airflow) to submit jobs to the Processing Engine (Spark). We solve this using a Docker-in-Docker approach, supported by a dedicated metadata database.

- **Socket Mounting:** We mount the host's `/var/run/docker.sock` into the Airflow container. This allows Airflow to "see" and control sibling containers.
- **Custom Image:** We build a custom Airflow image that includes the Docker CLI, enabling the execution of `docker exec spark-master ...` commands directly from our DAGs.
- **Postgres (Metadata Store):** We deploy a dedicated PostgreSQL container to serve as Airflow's backend. This database stores the state of every task, DAG run history, and variable configurations, ensuring persistence across container restarts.

```
FROM apache/airflow:2.7.1

USER root

# Install Docker CLI to allow triggering commands on Spark Master
RUN apt-get update && \
    apt-get install -y docker.io && \
    apt-get clean

USER airflow
```

```
postgres:
  image: postgres:14
  container_name: airflow-postgres
  environment:
    POSTGRES_USER: airflow
    POSTGRES_PASSWORD: airflow
    POSTGRES_DB: airflow
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD", "pg_isready", "-U", "airflow"]
    interval: 5s
    retries: 5

airflow:
  build: src/dags # Uses the custom Dockerfile defined above
  container_name: airflow
  depends_on:
    - postgres
    - spark-master
  environment:
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    # Points to the internal Spark Master container
    SPARK_MASTER: spark://spark-master:7077
  volumes:
    - ./src/dags:/opt/airflow/dags
    - ./src:/opt/airflow/src
    # CRITICAL: Mount host docker socket for Docker-in-Docker control
    - /var/run/docker.sock:/var/run/docker.sock
  ports:
    - "8082:8080" # Maps Airflow UI to port 8082
  command: >
    bash -c "
      airflow db migrate &&
```

```
      airflow users create --username admin --password admin ... || true &&
      airflow webserver & airflow scheduler
    "
```

4.5.2 Pipeline 1: Periodic ETL & Inference

The primary DAG (`weather_etl_pipeline`) is the heartbeat of our batch processing layer. Scheduled to run every 20 minutes, it orchestrates the flow of data from raw archival to actionable insights.

This pipeline enforces a strict dependency chain: Silver → Gold → Prediction. This sequential execution guarantees that the machine learning models always infer based on the freshest, fully aggregated data available.

The DAG consists of three sequentially executed tasks, all utilizing the `BashOperator` to trigger Spark jobs via the `docker exec` pattern:

1. **transform_silver:**
   - **Function:** Triggers the `bronze_to_silver.py` Spark job.
   - **Action:** Reads accumulated raw data from Bronze, applies physical limit validation, performs temporal imputation, and splits data into Fact/Dimension tables.
   - **Resource:** configured with 1 Core, 1GB Memory. Uses `-py-files` to ship shared schema definitions to workers.
2. **aggregate_gold:**
   - **Function:** Triggers the `silver_to_gold.py` Spark job.
   - **Action:** Consumes cleaned Silver data to compute business aggregates (Daily, Weekly) and generates ML features (sliding windows, cyclical time) for the next stage.
   - **Resource:** Configured with 1 Core, 1GB Memory. Depends strictly on the successful completion of `transform_silver`.
3. **predict_weather:**
   - **Function:** Triggers the `predict_weather.py` inference script.
   - **Action:** Loads station-specific Random Forest models from HDFS, retrieves the latest feature vectors directly from the **Silver Layer (HDFS)**, and generates forecasts for $T+1$ (Tomorrow).
   - **Resource:** Configured with 1 Core, 1GB Memory.

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {
  'owner': 'data-engineer',
  'retries': 1,
  'retry_delay': timedelta(minutes=5),
}

with DAG(
  'weather_etl_pipeline',
  default_args=default_args,
  description='Periodic Weather ETL (Bronze -> Silver -> Gold -> Forecast)',
  schedule_interval='*/20 * * * *', # Runs every 20 minutes
  start_date=datetime(2023, 11, 30),
  catchup=False,
  tags=['spark', 'weather', 'etl']
```

```
) as dag:

  # 1. SILVER JOB (Clean & Normalize)
  # Reads Bronze files, applies schema/limits, writes to Silver Fact/Dim
  silver_transformation = BashOperator(
    task_id='transform_silver',
    bash_command="""
    docker exec spark-master /opt/spark/bin/spark-submit \
    --master spark://spark-master:7077 \
    --deploy-mode client \
    --conf spark.cores.max=1 \
    --conf spark.executor.memory=1024m \
    --py-files /opt/spark/src/streaming/schema.py \
    /opt/spark/src/batch/bronze_to_silver.py
    """
  )

  # 2. GOLD JOB (Aggregate & Feature Engineering)
  # Reads Silver, calculates Daily/Weekly stats, generates ML features
  gold_aggregation = BashOperator(
    task_id='aggregate_gold',
    bash_command="""
    docker exec spark-master /opt/spark/bin/spark-submit \
    --master spark://spark-master:7077 \
    --deploy-mode client \
    --conf spark.cores.max=1 \
    --conf spark.executor.memory=1024m \
    /opt/spark/src/batch/silver_to_gold.py
    """
  )

  # 3. PREDICTION JOB (Inference)
  # Loads models, reads HDFS Silver data, predicts tomorrow's weather
  predict_weather = BashOperator(
    task_id='predict_weather',
    bash_command="""
    docker exec -u 0 spark-master /opt/spark/bin/spark-submit \
    --master spark://spark-master:7077 \
    --deploy-mode client \
    --conf spark.cores.max=1 \
    --conf spark.executor.memory=1024m \
    /opt/spark/src/ml/predict_weather.py
    """
  )

  # Define Dependencies: Enforce sequential execution
  silver_transformation >> gold_aggregation >> predict_weather
```

Listing 5: Primary ETL DAG (weather_etl_pipeline.py)

4.5.3 Pipeline 2: Model Retraining

The secondary DAG (`weather_model_training`) is designed to address seasonal variations. Weather patterns change over time, so a model trained on summer data will perform poorly in winter.

This pipeline runs once daily (`@daily`) to retrain the Random Forest models using the complete historical dataset accumulated in the Gold layer. By constantly refreshing the models with the latest ground truth, we ensure the forecasts remain accurate year-round.

The DAG consists of a single, resource-intensive task:

- **train_model:**

- **Function:** Triggers the `train_model.py` Spark job.
- **Action:** Reads the full history from `/weather/gold/ml_features`, generates new training labels (supervised learning), and trains 4 distinct Random Forest models for every single weather station.
- **Resources:** Unlike the ETL jobs, this task is configured with higher resource limits (2 cores, 1GB Memory each) to handle the parallel training of hundreds of models across the cluster.

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {
  'owner': 'data-engineer',
  'retries': 1,
  'retry_delay': timedelta(minutes=10),
}

with DAG(
  'weather_model_training',
  default_args=default_args,
  description='Retrains the Weather Forecast Model (Daily)',
  schedule_interval='@daily',
  start_date=datetime(2023, 11, 30),
  catchup=False,
  tags=['spark', 'mlops', 'training'],
) as dag:

  # 1. MODEL TRAINING TASK
  # Allocates more resources (2 cores) for intensive ML training
  train_model = BashOperator(
    task_id='train_model',
    bash_command="""
    docker exec spark-master /opt/spark/bin/spark-submit \
    --master spark://spark-master:7077 \
    --deploy-mode client \
    --conf spark.cores.max=2 \
    --conf spark.executor.memory=1024m \
    --conf spark.driver.memory=1024m \
    /opt/spark/src/ml/train_model.py
    """
  )
```

4.5.4 Airflow Verification

To verify the orchestration layer, we access the Airflow Web UI running on port 8082. This interface provides a real-time view of our pipeline status, scheduling configurations, and execution history.

As shown in Figure 13, both Directed Acyclic Graphs (DAGs) have been successfully deployed and are in an **Active** state (toggled ON).

The dashboard confirms our specific scheduling requirements:

- **weather_etl_pipeline:** Shows a schedule of `*/20 * * * *` (Every 20 minutes). The "Recent Tasks" view indicates a series of successful runs (green circles), verifying that the Bronze → Silver → Gold dependency chain is executing reliably.
- **weather_model_training:** Shows a schedule of `@daily`. The execution history confirms it has run successfully 2 times, proving that the Machine Learning retraining job triggers correctly

once per day.

The successful completion of these tasks also implicitly verifies our infrastructure configuration. Since the Airflow tasks are `BashOperators` running `docker exec`, their success proves that the Airflow container has valid access to the host's Docker socket and can successfully submit jobs to the `spark-master` container.

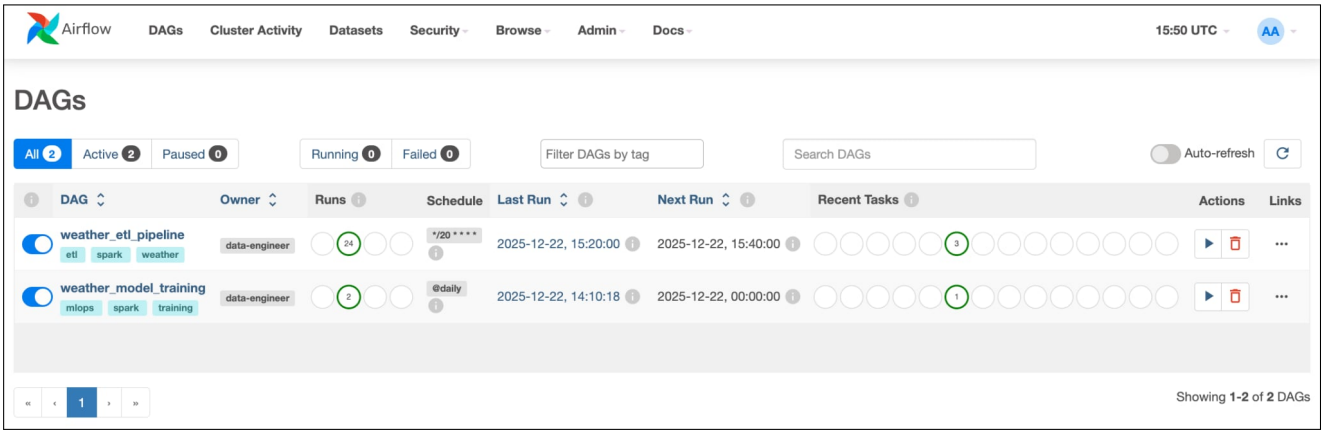

Figure 13: Airflow Dashboard: Verification of active DAGs, schedules, and successful run history.

4.6 Streamlit

To make the processed data actionable for end-users, we developed a responsive dashboard using Streamlit. This application unifies real-time streams from Cassandra with historical analytics from HDFS.

4.6.1 Dashboard Interface Verification

The following figures demonstrate the fully operational dashboard, highlighting the integration of the Speed and Batch layers.

a. Network Overview

Figure 14 shows the landing page. The application successfully loads station metadata from the Silver Layer (or Cassandra fallback) to populate an interactive map. Users can click any red marker to filter the downstream analytics for that specific location.

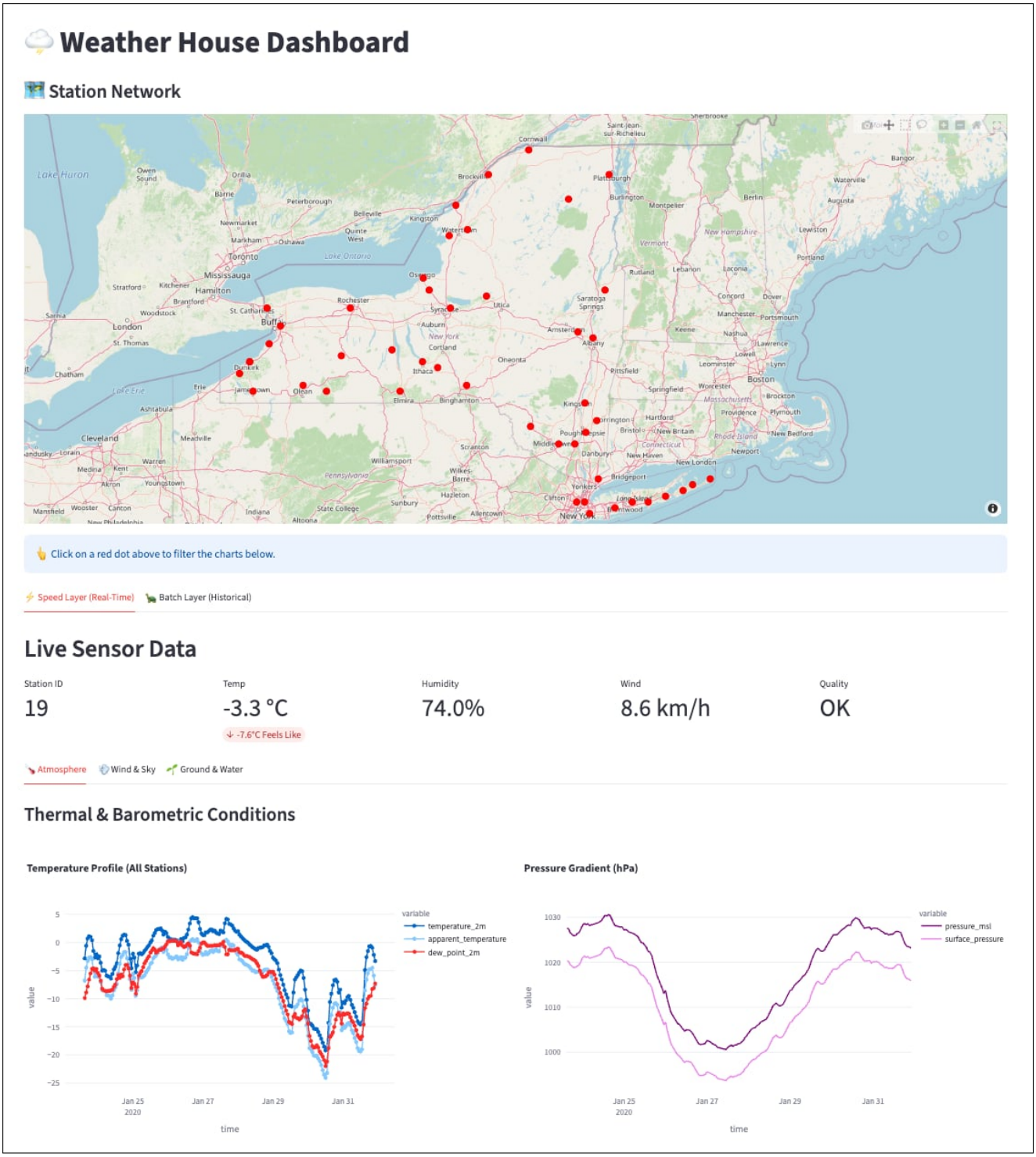

Figure 14: Dashboard Overview: Interactive map of the weather station network.

b. Real-Time Analytics

Once a station is selected, the "Speed Layer" tab provides sub-second visualization of sensor readings. The detailed breakdown is split across three thematic tabs to organize the 30+ distinct metrics.

Atmosphere Tab: Figure 15 displays thermal and barometric trends. Note the "AI Forecast" section at the top, which pulls the latest prediction (Tomorrow's Forecast) from the Gold layer to display alongside live data.

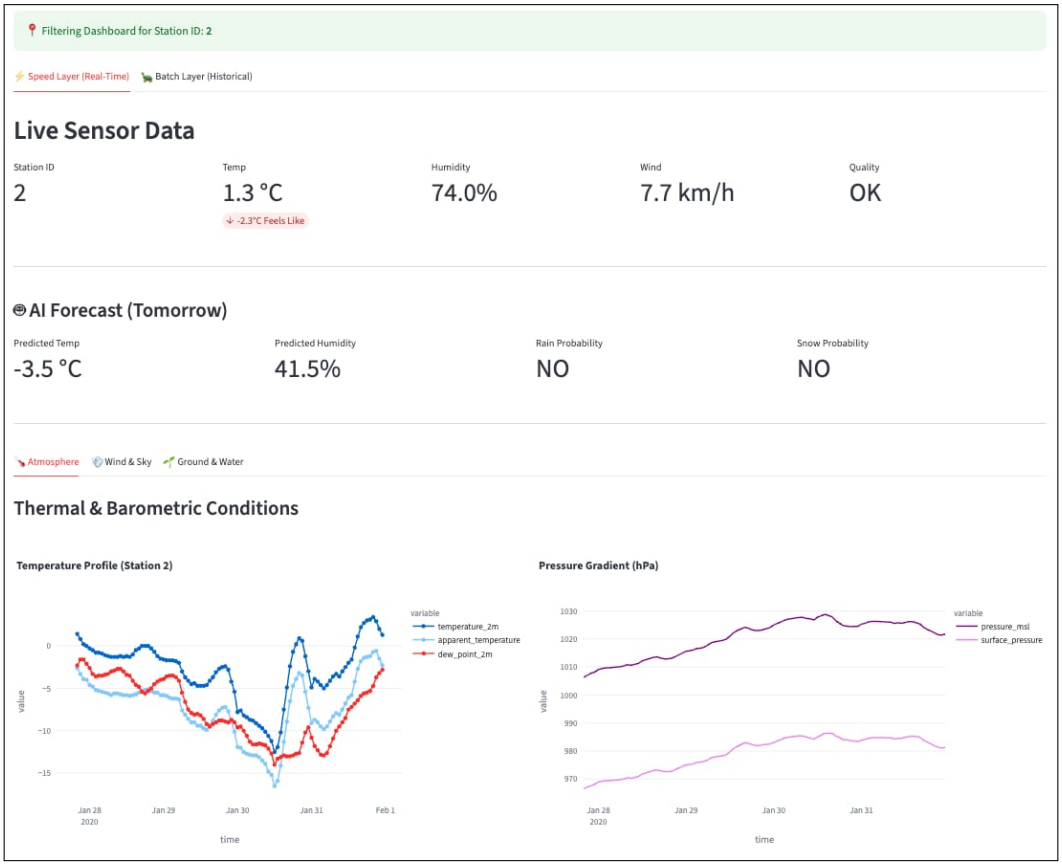

Figure 15: Atmosphere Tab: Live temperature/pressure data + AI Forecasts.

Wind & Sky Tab: Figure 16 visualizes wind vectors, distinguishing between sustained wind speeds and sudden gusts.

Figure 16: Wind Profile: Visualizing gusts (size) and direction (color).

Ground & Water Tab: Figure 17 tracks precipitation types (Rain vs. Snow) and soil physics (Mois-

ture/Temperature at various depths).

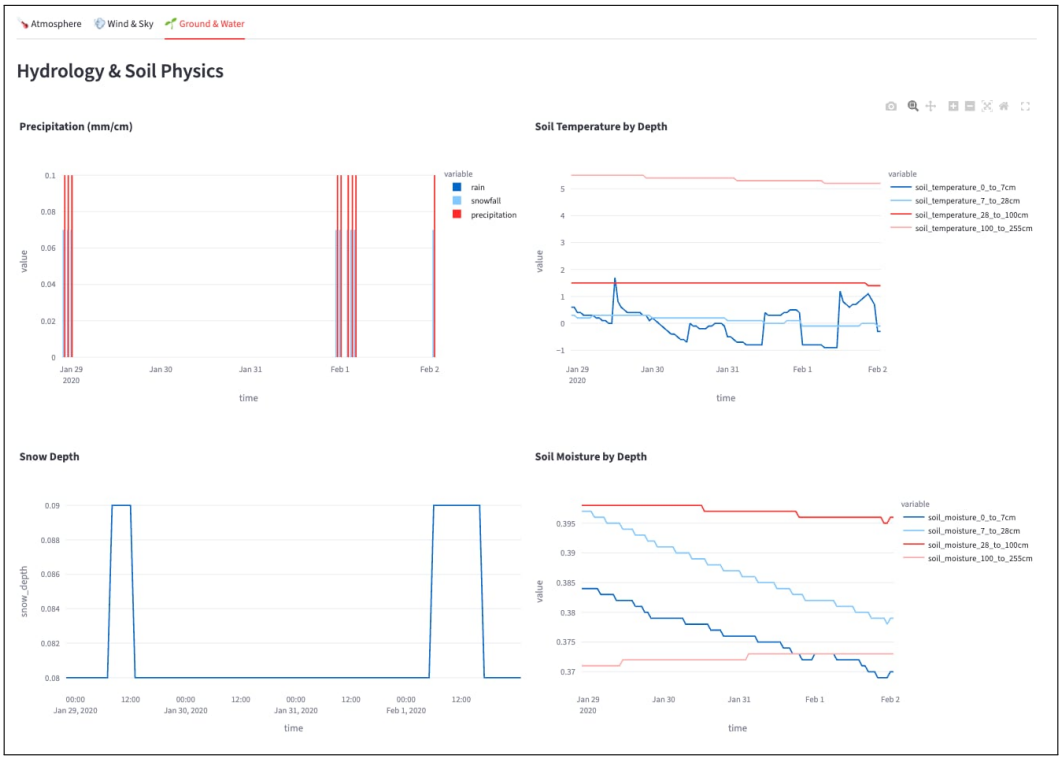

Figure 17: Ground Physics: Soil moisture saturation and precipitation classification.

c. Historical Analytics

Figure 18 confirms the integration with the Gold Layer. This view reads `daily_summary` and `weekly_summary` Parquet files from HDFS, allowing users to analyze long-term trends without querying the raw sensor logs.

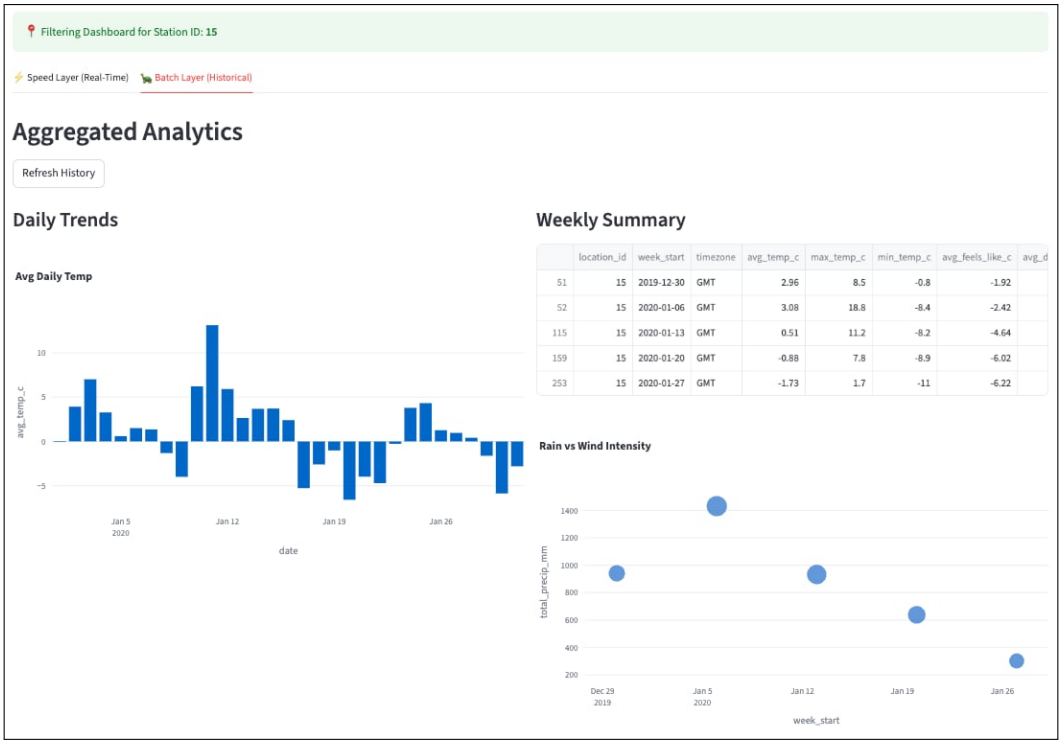

Figure 18: Batch Analytics: Historical trends derived strictly from the Gold Layer.

5. Conclusion

Through this project, our team has successfully explored, researched, and built a comprehensive Big Data pipeline using the Kappa Architecture. By integrating core technologies such as Kafka for ingestion, HDFS for storage, Spark for processing, and Cassandra for the serving layer, we created a system capable of handling meteorological data in both real-time and batch modes.

We successfully implemented the "Medallion" architecture (Bronze, Silver, Gold), ensuring that raw data is systematically cleaned, aggregated, and transformed into actionable Machine Learning insights. This project not only reinforced our technical skills in distributed systems but also demonstrated the practical application of data engineering principles to solve real-world problems.

While the current system is functional, we aim to further develop the pipeline to meet more diverse user needs in the future.

6. Acknowledgments

To complete this report, we would like to express our deepest gratitude to our instructor, Dr. Tran Viet Trung. His invaluable and inspiring lectures in the course *"Big Data Analysis and Processing"* provided us with the solid theoretical foundation and practical guidance necessary to navigate these complex technologies. We sincerely appreciate the knowledge and enthusiasm he has shared with us throughout this semester.

7. Lessons Learned

Throughout the development of the Weather House pipeline, we encountered several architectural challenges. Below are the key lessons learned.

7.1 Lesson 1: Data Quality & Ingestion Reliability

Problem: Real-time sensor data is notoriously unreliable. Our streams frequently contained duplicates from Kafka, missing values, and physically impossible outliers (e.g., temperature spikes > 60°C) which would severely skew our downstream Machine Learning models.

Solution: Instead of discarding imperfect data, we implemented a data imputation strategy at the silver layer. We first deduplicated records based on their unique ID and timestamp. Then, rather than dropping outliers, we masked them as null values and applied linear interpolation to fill the gaps. This ensured we maintained a continuous time-series history essential for calculating rolling window features.

Key Takeaway: For time-series data, continuity is often more important than perfection. Using interpolation to repair data is preferable to dropping rows, which creates gaps that break sliding-window calculations.

7.2 Lesson 2: Container Networking & Communication

Problem: Initial attempts to connect independent services (Kafka, Spark, Cassandra) failed because containers were isolated by default. Using `localhost` addresses inside containers proved ineffective as it refers to the container itself, preventing services like Airflow from triggering Spark jobs.

Solution: We consolidated all services into a single `docker-compose` file, leveraging the default

Docker bridge network. This allowed us to use service names (e.g., `spark-master`, `kafka1`) as hostnames for automatic DNS resolution.

Key Takeaway: Never use `localhost` in containerized environments. A shared Docker network with service-name addressing enables seamless service discovery without hardcoded IPs.

7.3 Lesson 3: Horizontal Scaling for Parallel Processing

Problem: The "Localized Learning" strategy requires training distinct Random Forest models for every station. A single Spark Worker node bottlenecked these parallel tasks, causing the daily training pipeline to exceed its time window. Vertical scaling (adding RAM/Cores) yielded minimal gains due to JVM limitations.

Solution: We implemented horizontal scaling by deploying a 3-node Spark cluster. The scheduler automatically distributed the training tasks across these nodes, reducing total execution time by approximately 60%.

Key Takeaway: Distributed frameworks like Spark are optimized for horizontal scaling. Adding worker nodes provides near-linear performance improvements for parallel workloads like multi-model training.

7.4 Lesson 4: Fault Tolerance via Data Replication

Problem: Running single instances of Kafka and HDFS meant that any container crash caused immediate data loss and pipeline failure. A single point of failure in the ingestion layer stopped the entire Speed Layer.

Solution: We implemented 3-way Replication across both layers. We deployed 3 Kafka Brokers with replication factor of 3 and 3 HDFS Datanodes.

Key Takeaway: Hardware unreliability is a given in Big Data. Software-level replication is essential for production stability, despite the trade-off of increased storage overhead.

8. Member Contribution

Member Name	Assigned Tasks & Responsibilities	Contribution
Nguyen Hai Dang	• Implemented Spark Structured Streaming for the Cold Path. • Developed Medallion ETL jobs (Bronze → Silver → Gold). • Built the Spark Machine Learning pipeline.	20%
Nguyen Hoang Quan	• Data gathering • Led the Docker Containerization and system infrastructure. • Collaborated on Medallion ETL jobs and data transformations.	20%
Phan Minh Hoa	• Implemented Spark Structured Streaming for the Hot Path. • Designed the Cassandra schema for near real-time serving. • Designed and built the Streamlit Dashboard.	20%
Nguyen Vo Ngoc Khue	• Developed the Python Producer code for sensor simulation. • Configured Kafka topics, partitions, and replication factors. • Collaborated on the Spark Machine Learning pipeline.	20%
Nguyen Tien Dat	• Developed Airflow DAGs for orchestration and scheduling. • Collaborated on the Streamlit Dashboard UI and visualizations. • Design presentation slide	20%

Table 1: Breakdown of team member responsibilities and contribution percentage.